Rapport på løsning Oblig 2 – IN2031
Utvidet språk med flyve-instruksjoner

Matias Nupen matiasnu@ifi.uio.no

12. oktoberber 2025

1. Introduksjon

Denne rapporten dokumenterer løsningen på den andre obligatoriske oppgaven i IN2031. Oppgaven er andre del av fire, som samlet skal resultere i et domene-spesifikt-språk (DSL). Jeg bruker hver oppgave som et delmål i prosjektet, med GitHub som versjonskontroll og utviklingsbrancher for å strukturere arbeidet. Koden i oblig 2 er basert på prekoden som ble gitt ut sammen med oppgaven.

Målet med denne andre delen er å utvide språket med flyve-instruksjoner som skal tolkes gjennom klassen `Interpreter`. Tidligere tok prosjektet inn enkle ord og utrykk i testfilene, som skulle parseres. Denne implementasjonen utvider prosjektet og skal kunne lese inn "mode", "statement" og "action", for så å kjøre disse i et "Program". Rapporten redegjør for implementasjonen, hvordan jeg testet løsningen, og hvilke utfordringer og mulige forbedringer som ble identifisert.

2. Implementasjon

Som i oblig 1, er programmet strukturert rundt en ANTLR-bygd parser og lexer, en `AST`, og en `Interpreter` som traverserer treet.

2.1. Grammatikk

Grammatikken som skulle utvide programmet ble gitt i oppgaven, og er definert i `AeroScript.g4`. Jeg valgte å benytte navnet "mode" etter melding sendt ut på semestersiden, men fikk ikke lov av ANTLR å bruke det navnet. Det ble derfor definert som `modeDecl`. "action" er definert for å ta hensyn til eventuelle modifiseringer gjennom "`FOR ... SECONDS`" eller "`AT SPEED ...`" for å sikre at uttrykket følger de grammatiske prinsippene som ble gitt i oppgaven.

2.2. AST og evaluering av Expressions

Mye av logikken som ble gitt ut med prekoden, og som i forrige oppgave ble definert har ikke endret seg betraktelig. I `OperationExpression.java` ble det lagt til case-variabler som brukt i utgitt test for å håndtere aritmetiske operasjoner. I testfilen var det brukt PLUS, MINUS, TIMES, og dette la jeg til som alternativer til SUM, SUB, MUL. `OperationExpression.java` ble også redefinert til å bruke de andre Expression-filene sine metoder. Ellers ble det gjort lite endringer på Expression-filene. NegNumberExpression måtte endres litt på – se dokumentasjon i fil.

2.3. AST og evaluering av Mode, Statement og Action

Logikken i Action baserer seg på prinsipper om arv og klassehierarki. Den abstrakte superklassen `Action.java` har en referanse til en `DroneState` (se pkt. 2.4) for å ha en referanse til dronen. Den har også to variabler av typen `NumberExpresssion` som beholder evt. antall sekunder eller hastighet. Dette for å ta hensyn til modifiseringene (se pkt. 2.1). For å holde riktig "state" på dronen besluttet jeg å først gjennomføre metoden `executeAction()`, for så å gjøre `consumeBattery()`. I `Action.java` gjennomfører disse kun logging gjennom `System.out.println(...)`, med unntak av sistnevnte metode, som også oppdaterer dronens batteri og sjekker om dronen har nok batteri/er tom.

Klassehierarkiet til Action ligger definert under filen `action_class_hierarchy.txt` og viser en illustrasjon av klassehierarkiet. I tillegg ligger det en detaljert beskrivelse av strukturen, som jeg ved hjelp av KI og Claude Sonnet 4-modellen fikk generert. I filen ligger også hvilken prompt jeg brukte. Jeg gikk over den genererte informasjonen etterpå, og var fornøyd med resultatet.

Jeg fant det logisk å bruke arv for å definere de ulike Action-klassene fordi det skaper en bedre struktur på programmet. Jeg fant fort ut at mange av metodene jeg begynte å skrive ble veldig like, og begynte da å tegne opp et hierarki som skulle hjelpe meg å løse det problemet. Alle har hver sin definisjon av hvordan batteriet på dronen brukes, hvordan dronens state oppfører seg, og lignende. Jeg er svært fornøyd med resultatet jeg kom frem til i utformingen av Action-klassene.

2.4. Implementasjon av Interpreter og DroneState

Klassen `Interpreter.java` tar det parserte programmet og går gjennom hver "Mode", hver "Statement", og hver "Action" og sørger for at modifikasjonene blir tatt hensyn til. Klassen sørger for at programmet kjører i rekkefølgen som er gitt, og holder programmets variabler i synk med kjøringen. I Runtime-mappen jobber den sammen med `DroneState.java`.

Jeg valgte å definere en singleton-basert klasse `DroneState.java` som holder på dronens State under en kjøring. Under kjøring vil dronen være utsatt for endring, og klassen sørger for å sentralisere dronens endringsbare tilstand. Denne lagrer og holder på ulike variabler som; posisjon, høyde, retning, total distanse og batteristatus.

3. Testing

Testsuiten består av to hoveddeler som har blitt utvidet fra den originale prekoden.

OperationExpressionTest.java tester alle binære og unære operasjoner systematisk. Det som har blitt utvidet fra prekoden er:

- Organiserte testklasser for aritmetikk (`BasicArithmeticTests`)
- Komplekse aritmetiske operasjoner (`ComplexArithmeticTests`)
- Spesialoperasjoner som tilfeldige tall og punkter (`SpecialOperationTests`)
- Tilfeller som dekker uforventet oppførsel (`EdgeCaseTests`)

InterpreterTest.java fokuserer på tolkefunksjonalitet og har blitt utviklet for å sjekke implementasjonen av alt som ble løst i oblig 2. Dette gjelder "Action", "Statement", "Mode", og kjøringen av disse gjennom Interpreter.java. Testprogrammet oppretter flere testkjøringer som:

- Verifiserer parsing av uttrykk gjennom Interpreter.java (`ExpressionParsingTests`)
- Tester håndtering av ulike "Modes" og overganger (`ModeTests`)
- Kontrollerer variabler og tilstandsendringer i DroneState (`VariableTests`)
- Kjører komplekse program for å teste helhetlig (`ComplexScenarioTests`)

Alle testene logger sine resultater i et oversiktelig format, som gjør debugging og verifisering enklere.

3.1. Begrensinger ved testing

- Feilhåndtering og tilfeller som dekker uforventet oppførsel. Det kunne blitt dekt et enda større omfang av negative utfall.
- Testene fokuserer hovedsakelig på enhetstesting av individuelle komponenter. Det kunne vært en mer omfattende integrasjonstesting mellom Interpreter, DroneState og de forskjellige Statement-typene.

4. Utfordringer og eventuelle problemer

I utviklingen fulgte jeg reglene som ble gitt ut i oppgaven, men eneste jeg gjorde var å endre verdien batteriet bruker på de ulike operasjonene. Dette gjøres i metoden `consumeBattery()` og bruker multiplikasjonsverdier fra hver av klassene. For eksempel er `DISTANCE_BATTERY_MULTIPLIER` = 0.05f, og ikke = 0.5f som gis i oppgaven. Dette fordi verdiene ble veldig høye når jeg brukte opprinnelig multiplikasjonsverdier. Batterikapasiteten er også satt til 100f, som betyr at det er mer hensiktsmessig å senke batterikostnaden.

5. Konklusjon

Løsningen utvider DSL-en med Mode/Statement/Action tolket av Interpreter, støttet av et ryddig arv-hierarki og en DroneState, og er verifisert gjennom enhetstester.